Applied Cryptography, Project 3

Exploring Block Cipher and the Meet in the Middle Attack

Group 47
Natania Jeffery (nj2153)
Joe Raymond (cjr10071)
Shreyas Sankpal (sss10093)
Enoch Sanyaolu (mos7641)

Problem 1: Modes of Operation

The cipher only reorders bits, and D puts them back:

$$E(b_1b_2b_3b_4) = (b_2b_3b_1b_4) \qquad D(y_1y_2y_3y_4) = (y_3y_1y_2y_4)$$

Plaintext: FOO = 0101 1110 1110, so $m_1 = 0101$ and $m_2 = m_3 = 1110$ (no padding needed).

2a: ECB

$c_i = E(m_i)$:

$c_1 = E(0101) = 1001 = J$
$c_2 = E(1110) = 1110 = O$
$c_3 = E(1110) = 1110 = O$

Ciphertext: **JOO**. Both O blocks give the same letter, so ECB shows the repeat.

2b: CBC with IV = 1010

$c_0 = IV$, $c_i = E(m_i \oplus c_{i-1})$:

$c_1 = E(0101 \oplus 1010) = E(1111) = 1111 = P$
$c_2 = E(1110 \oplus 1111) = E(0001) = 0001 = B$
$c_3 = E(1110 \oplus 0001) = E(1111) = 1111 = P$

Ciphertext: **PBP**, sent as **KPBP** with the IV first (same as in the class example).

2c: CTR with ctr = 1010

$c_i = m_i \oplus E(ctr_i)$. As in the class example, the first block uses ctr itself, so $ctr_1 = 1010$, $ctr_2 = 1011$, $ctr_3 = 1100$:

$c_1 = 0101 \oplus E(1010) = 0101 \oplus 0110 = 0011 = D$
$c_2 = 1110 \oplus E(1011) = 1110 \oplus 0111 = 1001 = J$
$c_3 = 1110 \oplus E(1100) = 1110 \oplus 1010 = 0100 = E$

Ciphertext: **DJE**, sent as **KDJE** with ctr first. CBC and CTR both hide the repeated O.

3a: Decryption

ECB: $m_i = D(c_i)$:

$m_1 = D(1001) = 0101 = F$
$m_2 = D(1110) = 1110 = O$
$m_3 = D(1110) = 1110 = O$

CBC: $m_i = D(c_i) \oplus c_{i-1}$, with $c_0 = IV$:

$m_1 = D(1111) \oplus 1010 = 1111 \oplus 1010 = 0101 = F$
$m_2 = D(0001) \oplus 1111 = 0001 \oplus 1111 = 1110 = O$
$m_3 = D(1111) \oplus 0001 = 1111 \oplus 0001 = 1110 = O$

CTR: $m_i = c_i \oplus E(ctr_i)$, the same keystream as in encryption, so D is not needed:

$m_1 = 0011 \oplus E(1010) = 0011 \oplus 0110 = 0101 = F$
$m_2 = 1001 \oplus E(1011) = 1001 \oplus 0111 = 1110 = O$
$m_3 = 0100 \oplus E(1100) = 0100 \oplus 1010 = 1110 = O$

All three decrypt to **FOO**. Checking in the notebook (In [3]):

```
ECB  sent: 1001 1110 1110       = JOO   decrypted: FOO
CBC  sent: 1010 1111 0001 1111  = KPBP  decrypted: FOO
CTR  sent: 1010 0011 1001 0100  = KDJE  decrypted: FOO
```

Problem 2: Meet-in-the-Middle Attack on the Mini Block Cipher

Task 1a: Mini block cipher

Built from the S-AES lecture (code: proj3-Group47.ipynb, in the appendix). The 16-bit state is the nibbles $n_0n_1n_2n_3$ as the matrix $[n_0\ n_2;\ n_1\ n_3]$.

Function	What it does
substitute	Lecture S-box on each nibble (inv_substitute uses the inverse table)
shift	Swaps the two bottom-row nibbles; it is its own inverse
mix	Each column times [1 4; 4 1] in GF(2^4) mod $x^4 + x + 1$ (inv_mix uses [9 2; 2 9])
add_round_key	XOR with the round key
key_expansion	$W_2 = W_0 \oplus g(W_1, 1000\ 0000)$, $W_3 = W_2 \oplus W_1$, $W_4 = W_2 \oplus g(W_3, 0011\ 0000)$, $W_5 = W_4 \oplus W_3$, where g = RotNib, SubNib, XOR RCON. $K_1 = W_2W_3$, $K_2 = W_4W_5$

The rounds follow this pseudo-code:

```
def encrypt_round1(k1, state):
    return add_round_key(mix(shift(substitute(state))), k1)


def encrypt_round2(k2, state):
```

```
    return add_round_key(shift(substitute(state)), k2)


def decrypt_round2(k2, state):
    return inv_substitute(shift(add_round_key(state, k2)))


def decrypt_round1(k1, state):
    return inv_substitute(shift(inv_mix(add_round_key(state, k1))))


def encrypt(key, plaintext):
    k1, k2 = key_expansion(key)
    return encrypt_round2(k2, encrypt_round1(k1, plaintext))


def decrypt(key, ciphertext):
    k1, k2 = key_expansion(key)
    return decrypt_round1(k1, decrypt_round2(k2, ciphertext))
```

Tests

Adding AddRoundKey(K) in front gives S-AES, which reproduces the lecture example (K = 0101 1001 0111 1010, P = "ZC") exactly:

```
K1 = W2 W3 : 1101 1100 1010 0110
K2 = W4 W5 : 0110 1100 1100 1010
S-AES(ZC)  : 1010 1001 1111 0110
```

Also, decrypt(encrypt(P)) = P for all 2^{16} plaintexts (In [9]). The mini cipher on the lecture input:

```
P             : 0101 1010 0100 0011
X (round 1)   : 0110 0011 0111 0111
C (round 2)   : 1110 1001 1001 0001
decrypt(K, C) : 0101 1010 0100 0011 = ZC
```

Task 1b: Ten plaintext-ciphertext pairs

Secret key K = 71d9, picked at random and then fixed. Each plaintext is two ASCII letters:

Plaintext	**HI**	**GO**	**OK**	**NO**	**UP**	**ME**	**WE**	**IT**	**ON**	**SO**
P (hex)	4849	474f	4f4b	4e4f	5550	4d45	5745	4954	4f4e	534f
C (hex)	02ec	1c62	afce	10b2	9d7a	b4f7	0c60	2f4b	4fc4	da3c

Task 2a: Meet-in-the-middle attack

The cipher is split after round 1, and the right key pair gives matching middle states:

$$X = \text{encrypt_round1}(K_1, P), \quad X' = \text{decrypt_round2}(K_2, C), \quad X = X'$$

The function mitm (steps a to c) stores X for all 2^{16} K_1 in a dictionary, then looks up X′ for all 2^{16} K_2. mitm_attack (step d) keeps only the key pairs that also fit the next pairs. K_1 and K_2 are searched independently, without the key expansion.

```python
def mitm(P, C):
    """Steps a to c: every (K1, K2) with encrypt_round1(K1, P) == decrypt_round2(K2, C)."""
    table = {}
    for k1 in range(1 << 16):
        table.setdefault(encrypt_round1(k1, P), []).append(k1)   # a
    matches = []
    for k2 in range(1 << 16):
        x = decrypt_round2(k2, C)                                # b
        matches += [(k1, k2) for k1 in table.get(x, [])]         # c
    return matches


def mitm_attack(pairs):
    """Step d: keep the matches of the first pair that also fit all the other pairs."""
    candidates = mitm(*pairs[0])
    counts = [len(candidates)]
    for P, C in pairs[1:]:
        candidates = [(k1, k2) for k1, k2 in candidates
                      if encrypt_round1(k1, P) == decrypt_round2(k2, C)]
        counts.append(len(candidates))
    return candidates, counts
```

Task 2b: Results

Key pairs left after each pair (In [12], 0.52 s in total):

Pairs used	**1**	**2**	**3**	**4 to 10**
(K_1, K_2) left	65,536	16	1	1

One pair leaves exactly 2^{16}, since X = Mix(Shift(Substitute(P))) $\oplus$ K_1 gives one K_1 for every K_2. Two pairs leave 16, not about 1: without Mix in round 2 the match splits into four per-nibble S-box conditions, each with an even number of solutions. Three pairs leave only K_1 = df06, K_2 = 6660.

Step e. Inverting the key expansion ($W_1 = W_2 \oplus W_3$, $W_0 = W_2 \oplus g(W_1, RCON_1)$) gives K = 71d9, the secret key. Any ciphertext can now be decrypted, for example a new message (In [13]):

```
recovered K = 71d9
key_expansion(K) == (K1, K2): True
same as SECRET_KEY: True

intercepted: 2ecb 16a7 aa3e b911 975a d6a0 7c6d
decrypted  : ATTACK AT DAWN
```

Task 3: Complexity

a) Key space: 16-bit key, so 2^{16} = 65,536 keys. The MITM above ignores the key expansion, so it covers all 2^{32} (K_1, K_2) pairs with about 2×2^{16} round computations.

b) MITM on the double cipher C = E(K_b, E(K_a, P)): encrypt P under all 2^{16} K_a and store the results, then decrypt C under all 2^{16} K_b and look them up:

$$2^{16} + 2^{16} = 2^{17} = 131{,}072 \text{ operations, with } 2^{16} \text{ table entries}$$

One pair leaves about 2^{16} false matches, so a second pair filters them (at most 2^{17} more operations). Measured (In [15]):

```
candidates after each pair: [65506, 3, 1, 1, 1, 1, 1, 1, 1, 1]
key found: 088ac4e1, correct: True
operations: 262,104 (2^18.00), time: 1.3 s
```

c) Exhaustive search: 2^{32} keys, each a double encryption, so $2^{33} \approx 8.6 \times 10^9$ operations (half on average). Estimated at 9.3 hours here, against 1.3 s for the MITM.

d) Trade-off:

	Exhaustive search	**Meet in the middle**
Operations	2^{33}	2^{17} (2^{18} with filtering)
Memory	almost none	2^{16} entries (256 KB packed)
Known pairs	2 to 3	2 to 3
Time here	about 9.3 hours (estimated)	1.3 s (measured)

The MITM trades memory for time. With M table entries it costs about $2^{32}/M$, so exhaustive search (M = 1) and the MITM ($M = 2^{16}$) are the two ends of one trade-off. For real keys memory is the limit (2^{56} entries for double DES), and double encryption adds only about one bit of security, which is why Triple DES uses three passes.

Task 4: Improving the mini block cipher

Is S-AES vulnerable? No. Its initial AddRoundKey(K_0) makes X = Mix(Shift(Substitute(P $\oplus$ K_0))) $\oplus$ K_1, so the forward half needs two round keys (2^{32} guesses, worse than brute force over 2^{16} keys). The key expansion does not help either: $K_0 = K$ fixes K_1 and K_2, so nothing is left to meet in the middle. The Task 2 attack on S-AES pairs (In [17]) finds no key pair after two pairs. The only weakness of S-AES is its 16-bit key.

Improvements:

1. Add the initial AddRoundKey (key whitening) as in S-AES, so round 1 is keyed.
2. Use more rounds, so each half of a MITM needs several round keys.
3. Use a nonlinear key schedule where every round key depends on all key bits, so the two halves cannot be guessed separately.
4. Use longer keys, and triple instead of double encryption: a MITM costs about 2^{k+1} on double but 2^{2k} on triple encryption.

References

1. Assignment handout, and class slides on Block Ciphers and Modes of Operation.
2. Meet-in-the-middle attack, Wikipedia (linked in the handout).
3. W. Diffie and M. E. Hellman, “Exhaustive Cryptanalysis of the NBS Data Encryption Standard”, Computer 10(6), 1977.
4. R. C. Merkle and M. E. Hellman, “On the Security of Multiple Encryption”, Communications of the ACM 24(7), 1981.

Appendix: Jupyter Notebook

Contents of proj3-Group47.ipynb with its output, included so this file covers both parts of the submission. The notebook is submitted separately as well.

Applied Cryptography, Project 3

Group 47

Natania Jeffery (nj2153)

Joe Raymond (cjr10071)

Shreyas Sankpal (sss10093)

Enoch Sanyaolu (mos7641)

Code for Problems 1 and 2. Problem 1 is done by hand in the report and only checked here. Standard library only.

In [1]:
```
import math
import time
```

Problem 1: Checking the hand calculations

Blocks are 4-bit strings (A = 0000 to P = 1111) and E(b1 b2 b3 b4) = (b2 b3 b1 b4).

In [2]:
```
LETTERS = "ABCDEFGHIJKLMNOP"


def to_bits(text):
    """Letters A-P -> list of 4-bit strings."""
    return [format(LETTERS.index(ch), "04b") for ch in text]


def to_letters(blocks):
    return "".join(LETTERS[int(b, 2)] for b in blocks)


def xor(a, b):
    return format(int(a, 2) ^ int(b, 2), "04b")


def E(b):
    """E(b1 b2 b3 b4) = (b2 b3 b1 b4)"""
    return b[1] + b[2] + b[0] + b[3]


def D(c):
    """Inverse of E: D(c1 c2 c3 c4) = (c3 c1 c2 c4)"""
    return c[2] + c[0] + c[1] + c[3]


def ecb_encrypt(m):
    return [E(x) for x in m]
```

```
def ecb_decrypt(c):
    return [D(y) for y in c]


def cbc_encrypt(m, iv):
    c = [iv]
    for x in m:
        c.append(E(xor(x, c[-1])))
    return c[1:]


def cbc_decrypt(c, iv):
    previous = [iv] + c[:-1]
    return [xor(D(y), p) for y, p in zip(c, previous)]


def ctr_keystream(ctr, n):
    """E(ctr), E(ctr + 1), ... The first block uses ctr itself, as in the class example."""
    return [E(format((int(ctr, 2) + i) % 16, "04b")) for i in range(n)]


def ctr_encrypt(m, ctr):
    return [xor(x, k) for x, k in zip(m, ctr_keystream(ctr, len(m)))]


ctr_decrypt = ctr_encrypt   # same keystream XOR, D is never used
```

CBC and CTR send the IV or counter as the first block, as in the class examples.

In [3]:
```
m = to_bits("FOO")
IV = CTR = "1010"

c_ecb, c_cbc, c_ctr = ecb_encrypt(m), cbc_encrypt(m, IV), ctr_encrypt(m, CTR)
results = [("ECB", c_ecb, ecb_decrypt(c_ecb)),
           ("CBC", [IV] + c_cbc, cbc_decrypt(c_cbc, IV)),
           ("CTR", [CTR] + c_ctr, ctr_decrypt(c_ctr, CTR))]

for mode, sent, back in results:
    blocks = " ".join(sent)
    print(f"{mode}  sent: {blocks:<20} = {to_letters(sent):<5} decrypted: {to_letters(back)}")
```

```
ECB  sent: 1001 1110 1110       = JOO   decrypted: FOO
CBC  sent: 1010 1111 0001 1111  = KPBP  decrypted: FOO
CTR  sent: 1010 0011 1001 0100  = KDJE  decrypted: FOO
```

Problem 2: Meet-in-the-middle attack on the mini block cipher

Task 1a: Mini block cipher

Building blocks from the S-AES lecture. The state n0 n1 n2 n3 is the matrix [[n0, n2], [n1, n3]], and arithmetic is in GF(2^4) mod x^4 + x + 1.

In [4]:
```
SBOX = [0x9, 0x4, 0xA, 0xB, 0xD, 0x1, 0x8, 0x5,
        0x6, 0x2, 0x0, 0x3, 0xC, 0xE, 0xF, 0x7]
INV_SBOX = [SBOX.index(x) for x in range(16)]


def gf_mult(a, b):
    """Multiply two nibbles in GF(2^4) mod x^4 + x + 1."""
    result = 0
```

```
    while b:
        if b & 1:
            result ^= a
        a <<= 1
        if a & 0x10:
            a ^= 0b10011
        b >>= 1
    return result


def to_nibbles(state):
    return [(state >> 12) & 0xF, (state >> 8) & 0xF, (state >> 4) & 0xF, state & 0xF]


def from_nibbles(n):
    return (n[0] << 12) | (n[1] << 8) | (n[2] << 4) | n[3]


def substitute(state):
    return from_nibbles([SBOX[x] for x in to_nibbles(state)])


def inv_substitute(state):
    return from_nibbles([INV_SBOX[x] for x in to_nibbles(state)])


def shift(state):
    """Swap the two nibbles of the bottom row. Doing it twice gives the state back."""
    n0, n1, n2, n3 = to_nibbles(state)
    return from_nibbles([n0, n3, n2, n1])


def mix(state):
    """Multiply each column by [[1, 4], [4, 1]]."""
    n0, n1, n2, n3 = to_nibbles(state)
    return from_nibbles([n0 ^ gf_mult(4, n1), gf_mult(4, n0) ^ n1,
                         n2 ^ gf_mult(4, n3), gf_mult(4, n2) ^ n3])


def inv_mix(state):
    """Multiply each column by the inverse matrix [[9, 2], [2, 9]]."""
    n0, n1, n2, n3 = to_nibbles(state)
    return from_nibbles([gf_mult(9, n0) ^ gf_mult(2, n1), gf_mult(2, n0) ^ gf_mult(9, n1),
                         gf_mult(9, n2) ^ gf_mult(2, n3), gf_mult(2, n2) ^ gf_mult(9, n3)])


def add_round_key(state, key):
    return state ^ key
```

Key expansion: K = W0 W1 gives K1 = W2 W3 and K2 = W4 W5.

In [5]:

```
RCON1, RCON2 = 0b10000000, 0b00110000


def g(w, rcon):
    """RotNib, SubNib, then XOR with the round constant (8-bit word)."""
    rotated = ((w << 4) | (w >> 4)) & 0xFF
    return ((SBOX[rotated >> 4] << 4) | SBOX[rotated & 0xF]) ^ rcon


def key_expansion(key):
    """16-bit key K -> round keys (K1, K2)."""
    w0, w1 = key >> 8, key & 0xFF
```

```
    w2 = w0 ^ g(w1, RCON1)
    w3 = w2 ^ w1
    w4 = w2 ^ g(w3, RCON2)
    w5 = w4 ^ w3
    return (w2 << 8) | w3, (w4 << 8) | w5
```

The two rounds, as in the pseudo-code. Shift is its own inverse.

In [6]:
```
def encrypt_round1(k1, state):
    return add_round_key(mix(shift(substitute(state))), k1)


def encrypt_round2(k2, state):
    return add_round_key(shift(substitute(state)), k2)


def decrypt_round2(k2, state):
    return inv_substitute(shift(add_round_key(state, k2)))


def decrypt_round1(k1, state):
    return inv_substitute(shift(inv_mix(add_round_key(state, k1))))


def encrypt(key, plaintext):
    k1, k2 = key_expansion(key)
    return encrypt_round2(k2, encrypt_round1(k1, plaintext))


def decrypt(key, ciphertext):
    k1, k2 = key_expansion(key)
    return decrypt_round1(k1, decrypt_round2(k2, ciphertext))
```

Check 1. S-AES is the mini cipher plus AddRoundKey(K) at the start, so it should reproduce the lecture example (K = 0101 1001 0111 1010, plaintext "ZC").

In [7]:
```
def bits(x):
    """16-bit value as four nibbles, e.g. 0101 1001 0111 1010."""
    return " ".join(format(x, "016b")[i:i + 4] for i in range(0, 16, 4))


def saes_encrypt(key, plaintext):
    """Full S-AES from the lecture: AddRoundKey(K) first, then the two rounds above."""
    k1, k2 = key_expansion(key)
    return encrypt_round2(k2, encrypt_round1(k1, add_round_key(plaintext, key)))


K_LECTURE = 0b0101_1001_0111_1010
P_LECTURE = int.from_bytes(b"ZC", "big")

k1, k2 = key_expansion(K_LECTURE)
print("K1 = W2 W3 :", bits(k1))
print("K2 = W4 W5 :", bits(k2))
print("S-AES(ZC)  :", bits(saes_encrypt(K_LECTURE, P_LECTURE)))
```

```
K1 = W2 W3 : 1101 1100 1010 0110
K2 = W4 W5 : 0110 1100 1100 1010
S-AES(ZC)  : 1010 1001 1111 0110
```

The same example through the mini cipher. X is the middle state the attack matches on.

In [8]:
```
x = encrypt_round1(k1, P_LECTURE)
c = encrypt_round2(k2, x)
p = decrypt(K_LECTURE, c)
print("P             :", bits(P_LECTURE))
print("X (round 1)   :", bits(x))
print("C (round 2)   :", bits(c))
print("decrypt(K, C) :", bits(p), "=", p.to_bytes(2, "big").decode())
```

```
P             : 0101 1010 0100 0011
X (round 1)   : 0110 0011 0111 0111
C (round 2)   : 1110 1001 1001 0001
decrypt(K, C) : 0101 1010 0100 0011 = ZC
```

Check 2. Decryption undoes encryption for all 2^16 blocks.

In [9]:
```
test_keys = [0x0000, K_LECTURE, 0xFFFF]
ok = all(decrypt(k, encrypt(k, p)) == p for k in test_keys for p in range(1 << 16))
print("decrypt(encrypt(P)) == P for all 65,536 plaintexts and all test keys:", ok)
```

```
decrypt(encrypt(P)) == P for all 65,536 plaintexts and all test keys: True
```

Task 1b: Ten plaintext-ciphertext pairs

The key was picked at random once and fixed so the results are reproducible. The attack only gets pairs.

In [10]:
```
SECRET_KEY = 0x71D9

words = ["HI", "GO", "OK", "NO", "UP", "ME", "WE", "IT", "ON", "SO"]
plaintexts = [int.from_bytes(w.encode(), "big") for w in words]
pairs = [(p, encrypt(SECRET_KEY, p)) for p in plaintexts]

print(" #  P    P (hex)  C (hex)  C (bits)")
for i, (w, (p, c)) in enumerate(zip(words, pairs), 1):
    print(f"{i:>2}  {w}   {p:04x}     {c:04x}     {bits(c)}")
```

```
 #  P    P (hex)  C (hex)  C (bits)
 1  HI   4849     02ec     0000 0010 1110 1100
 2  GO   474f     1c62     0001 1100 0110 0010
 3  OK   4f4b     afce     1010 1111 1100 1110
 4  NO   4e4f     10b2     0001 0000 1011 0010
 5  UP   5550     9d7a     1001 1101 0111 1010
 6  ME   4d45     b4f7     1011 0100 1111 0111
 7  WE   5745     0c60     0000 1100 0110 0000
 8  IT   4954     2f4b     0010 1111 0100 1011
 9  ON   4f4e     4fc4     0100 1111 1100 0100
10  SO   534f     da3c     1101 1010 0011 1100
```

Task 2a: Meet-in-the-middle attack

Steps a to c: store X = encrypt_round1(K1, P) for every K1, then look up X' = decrypt_round2(K2, C) for every K2. Step d: drop key pairs that fail the next pairs. K1 and K2 are searched independently, without the key expansion.

In [11]:
```
def mitm(P, C):
    """Steps a to c: every (K1, K2) with encrypt_round1(K1, P) == decrypt_round2(K2, C)."""
    table = {}
    for k1 in range(1 << 16):
        table.setdefault(encrypt_round1(k1, P), []).append(k1)   # a
    matches = []
    for k2 in range(1 << 16):
        x = decrypt_round2(k2, C)                                # b
```

```
        matches += [(k1, k2) for k1 in table.get(x, [])]         # c
    return matches


def mitm_attack(pairs):
    """Step d: keep the matches of the first pair that also fit all the other pairs."""
    candidates = mitm(*pairs[0])
    counts = [len(candidates)]
    for P, C in pairs[1:]:
        candidates = [(k1, k2) for k1, k2 in candidates
                      if encrypt_round1(k1, P) == decrypt_round2(k2, C)]
        counts.append(len(candidates))
    return candidates, counts
```

Task 2b: Results

In [12]:
```
start = time.perf_counter()
key_pairs, counts = mitm_attack(pairs)
print(f"attack time: {time.perf_counter() - start:.2f} s\n")

print("pairs used  (K1, K2) left")
for n, count in enumerate(counts, 1):
    print(f"{n:>10}  {count:>12,}")
print("\nkey pairs left:", [f"K1 = {k1:04x}, K2 = {k2:04x}" for k1, k2 in key_pairs])
```

```
attack time: 0.52 s

pairs used  (K1, K2) left
         1       65,536
         2           16
         3            1
         4            1
         5            1
         6            1
         7            1
         8            1
         9            1
        10            1

key pairs left: ['K1 = df06, K2 = 6660']
```

One pair leaves 2^16 key pairs, two leave 16 and three leave one.

Step e. Running the key expansion backwards from K1 gives K. Then any ciphertext can be decrypted, here a new message.

In [13]:
```
def invert_key_expansion(k1):
    """K1 = W2 W3 -> K = W0 W1."""
    w2, w3 = k1 >> 8, k1 & 0xFF
    w1 = w2 ^ w3
    w0 = w2 ^ g(w1, RCON1)
    return (w0 << 8) | w1


k1, k2 = key_pairs[0]
recovered_key = invert_key_expansion(k1)
print(f"recovered K = {recovered_key:04x}")
print("key_expansion(K) == (K1, K2):", key_expansion(recovered_key) == (k1, k2))
print("same as SECRET_KEY:", recovered_key == SECRET_KEY)

message = "ATTACK AT DAWN"
blocks = [int.from_bytes(message[i:i + 2].encode(), "big") for i in range(0, len(message), 2)]
```

```
intercepted = [encrypt(SECRET_KEY, b) for b in blocks]
recovered = b"".join(decrypt_round1(k1, decrypt_round2(k2, c)).to_bytes(2, "big")
                     for c in intercepted)
print("\nintercepted:", " ".join(f"{c:04x}" for c in intercepted))
print("decrypted  :", recovered.decode())
```

```
recovered K = 71d9
key_expansion(K) == (K1, K2): True
same as SECRET_KEY: True

intercepted: 2ecb 16a7 aa3e b911 975a d6a0 7c6d
decrypted  : ATTACK AT DAWN
```

Task 3: Double mini cipher

The first 16 key bits go to the first mini cipher and the last 16 to the second. One mini cipher encryption or decryption counts as one operation.

In [14]:
```
def double_encrypt(key32, plaintext):
    return encrypt(key32 & 0xFFFF, encrypt(key32 >> 16, plaintext))


def double_mitm(pairs):
    """MITM on the double mini cipher. Returns (keys left, candidate counts, operations)."""
    P, C = pairs[0]
    table = {}
    for ka in range(1 << 16):
        table.setdefault(encrypt(ka, P), []).append(ka)
    candidates = []
    for kb in range(1 << 16):
        candidates += [(ka, kb) for ka in table.get(decrypt(kb, C), [])]
    ops, counts = 2 << 16, [len(candidates)]
    for P, C in pairs[1:]:
        ops += 2 * len(candidates)   # one encryption and one decryption per candidate
        candidates = [(ka, kb) for ka, kb in candidates if encrypt(ka, P) == decrypt(kb, C)]
        counts.append(len(candidates))
    return [(ka << 16) | kb for ka, kb in candidates], counts, ops
```

In [15]:
```
SECRET_KEY32 = 0x088AC4E1
double_pairs = [(p, double_encrypt(SECRET_KEY32, p)) for p in plaintexts]

start = time.perf_counter()
keys, counts, ops = double_mitm(double_pairs)
mitm_time = time.perf_counter() - start

print("candidates after each pair:", counts)
print(f"key found: {keys[0]:08x}, correct: {keys == [SECRET_KEY32]}")
print(f"operations: {ops:,} (2^{math.log2(ops):.2f}), time: {mitm_time:.1f} s")
```

```
candidates after each pair: [65506, 3, 1, 1, 1, 1, 1, 1, 1, 1]
key found: 088ac4e1, correct: True
operations: 262,104 (2^18.00), time: 1.3 s
```

Exhaustive search over 2^32 keys would take hours, so I timed 2^16 keys and scaled up.

In [16]:
```
P, C = double_pairs[0]
start = time.perf_counter()
fits = sum(double_encrypt(key32, P) == C for key32 in range(1 << 16))
per_key = (time.perf_counter() - start) / (1 << 16)
exhaustive_time = per_key * 2**32

print(f"timed 2^16 keys, {fits} of them fit the first pair")
print(f"all keys: 2^32 = {2**32:,}, operations: 2^33 = {2**33:,}")
```

```
print(f"estimated time: {exhaustive_time / 3600:.1f} hours")
print(f"the MITM was {exhaustive_time / mitm_time:,.0f} times faster")
```

```
timed 2^16 keys, 1 of them fit the first pair
all keys: 2^32 = 4,294,967,296, operations: 2^33 = 8,589,934,592
estimated time: 9.3 hours
the MITM was 26,159 times faster
```

Task 4: Does the attack work on S-AES?

Same attack and key, but the pairs come from S-AES.

In [17]:
```
saes_pairs = [(p, saes_encrypt(SECRET_KEY, p)) for p in plaintexts]
_, counts = mitm_attack(saes_pairs)
print("candidates after each pair:", counts)
```

```
candidates after each pair: [65536, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

No key pair survives: with the first AddRoundKey the middle state depends on K, not only on K1.